

readable() Returns True if the file stream can be read from.
readline(n=-1) Read and return one line from the file. Reads in at most n bytes if specified.
readlines(n=-1) Read and return a list of lines from the file. Reads in at most n bytes/characters if specified.
seek(offset, from=SEEK_SET) Change the file position to offset bytes, in reference to from (start, current, end).
seekable() Returns True if the file stream supports random access.
tell() Returns the current file location.
truncate(size=None) Resize the file stream to size bytes. If size is not specified, resize to current location.
writable() Returns True if the file stream can be written to.
write(s) Write string s to the file and return the number of characters written.
writelines(lines) Write a list of lines to the file.

UNIT IV

IoT PHYSICAL DEVICES AND ENDPOINTS

IoT Device

A "Thing" in Internet of Things (IoT) can be any object that has a unique identifier and which can send/receive data (including user data) over a network (e.g., smart phone, smartTV, computer, refrigerator, car, etc.).

- IoT devices are connected to the Internet and send information about themselves or about their surroundings (e.g. information sensed by the connected sensors) over a network (to other devices or servers/storage) or allow actuation upon the physical entities/environment around them remotely.

IoT Device Examples

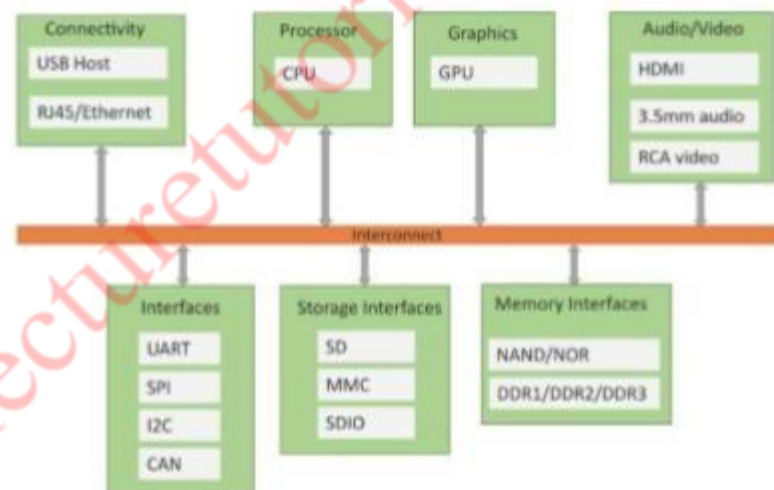
A home automation device that allows remotely monitoring the status of appliances and controlling the appliances. • An industrial machine which sends information about its operation and health monitoring data to a server. • A car which sends information about its location to a

cloud-based service. • A wireless-enabled wearable device that measures data about a person such as the number of steps walked and sends the data to a cloud-based service.

Basic building blocks of an IoT Device

1. **Sensing:** Sensors can be either on-board the IoT device or attached to the device.
2. **Actuation:** IoT devices can have various types of actuators attached that allow taking actions upon the physical entities in the vicinity of the device.
3. **Communication:** Communication modules are responsible for sending collected data to other devices or cloud-based servers/storage and receiving data from other devices and commands from remote applications.
4. **Analysis & Processing:** Analysis and processing modules are responsible for making sense of the collected data.

Block diagram of an IoT Device

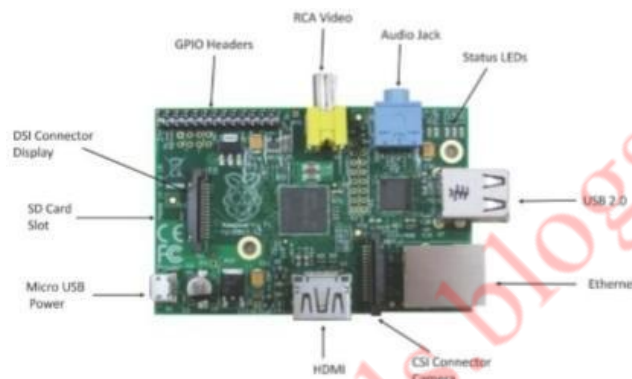


Exemplary Device: Raspberry Pi

Raspberry Pi is a low-cost mini-computer with the physical size of a credit card. Raspberry Pi runs various flavors of Linux and can perform almost all tasks that a normal desktop computer can do. Raspberry Pi also allows interfacing sensors and actuators through the general purpose

I/O pins. Since Raspberry Pi runs Linux operating system, it supports Python "out of the box". Raspberry Pi is a low-cost mini-computer with the physical size of a credit card. Raspberry Pi runs various flavors of Linux and can perform almost all tasks that a normal desktop computer can do. Raspberry Pi also allows interfacing sensors and actuators through the general purpose I/O pins. Since Raspberry Pi runs Linux operating system, it supports Python "out of the box".

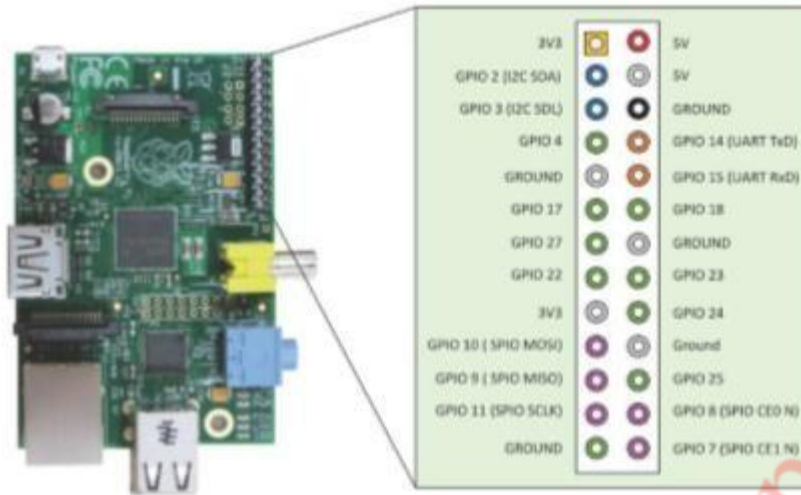
Raspberry Pi



Linux on Raspberry Pi

1. Raspbian: Raspbian Linux is a Debian Wheezy port optimized for RaspberryPi.
2. Arch: Arch is an Arch Linux port for AMDdevices.
3. Pidora: Pidora Linux is a Fedora Linux optimized for RaspberryPi.
4. RaspBMC: RaspBMC is an XBMC media-center distribution for RaspberryPi.
5. OpenELEC: OpenELEC is a fast and user-friendly XBMC media-centerdistribution.
6. RISC OS: RISC OS is a very fast and compact operatingsystem.

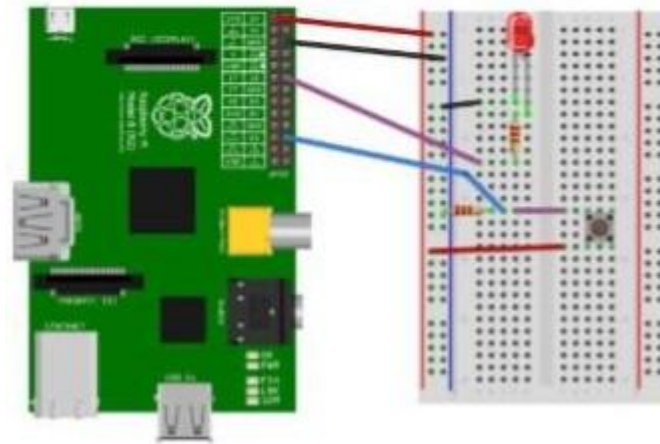
Raspberry Pi GPIO



Raspberry Pi Interfaces

1. **Serial:** The serial interface on Raspberry Pi has receive (Rx) and transmit (Tx) pins for communication with serial peripherals.
2. **SPI:** Serial Peripheral Interface (SPI) is a synchronous serial data protocol used for communicating with one or more peripheral devices.
3. **I2C:** The I2C interface pins on Raspberry Pi allow you to connect hardware modules. I2C interface allows synchronous data transfer with just two pins - SDA (data line) and SCL (clockline).

Raspberry Pi Example: Interfacing LED and switch with Raspberry Pi



```
from time import sleep
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
#Switch Pin GPIO.setup(25,GPIO.IN)
#LEDPin
GPIO.setup(18,GPIO.OUT)
state=False

def toggleLED(pin):
    state = not state
    GPIO.output(pin,state)

while True:
    try:
        if (GPIO.input(25) == True):
            toggleLED(pin)
            sleep(.01)
```

```
exceptKeyboardInterrupt:
```

```
    exit()
```

Other Devices

1. pcDuino
2. BeagleBoneBlack
3. Cubieboard

UNIT V